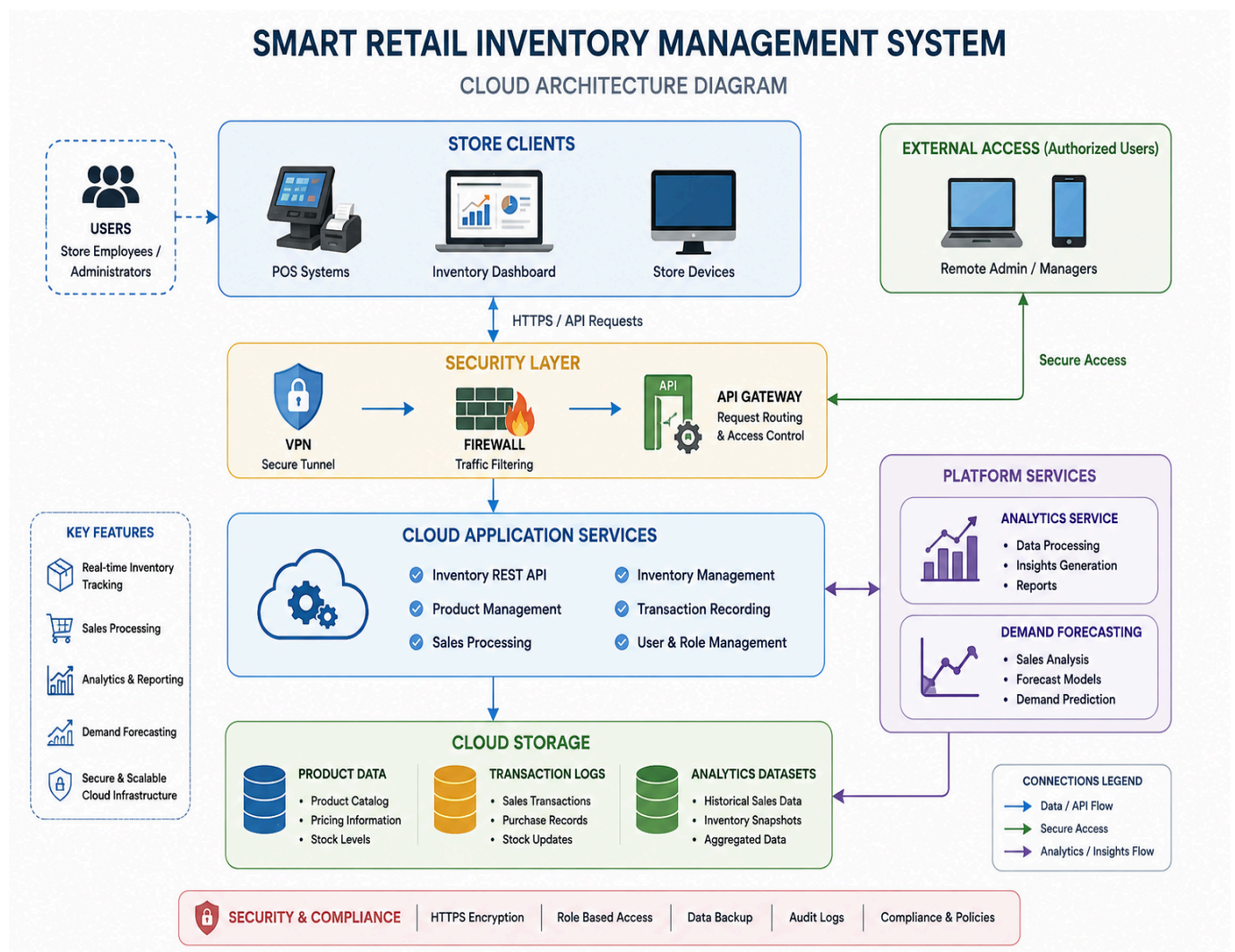


CASE STUDY 1

SMART RETAIL INVENTORY MANAGEMENT SYSTEM

1. ARCHITECTURE DIAGRAM



Architecture Workflow

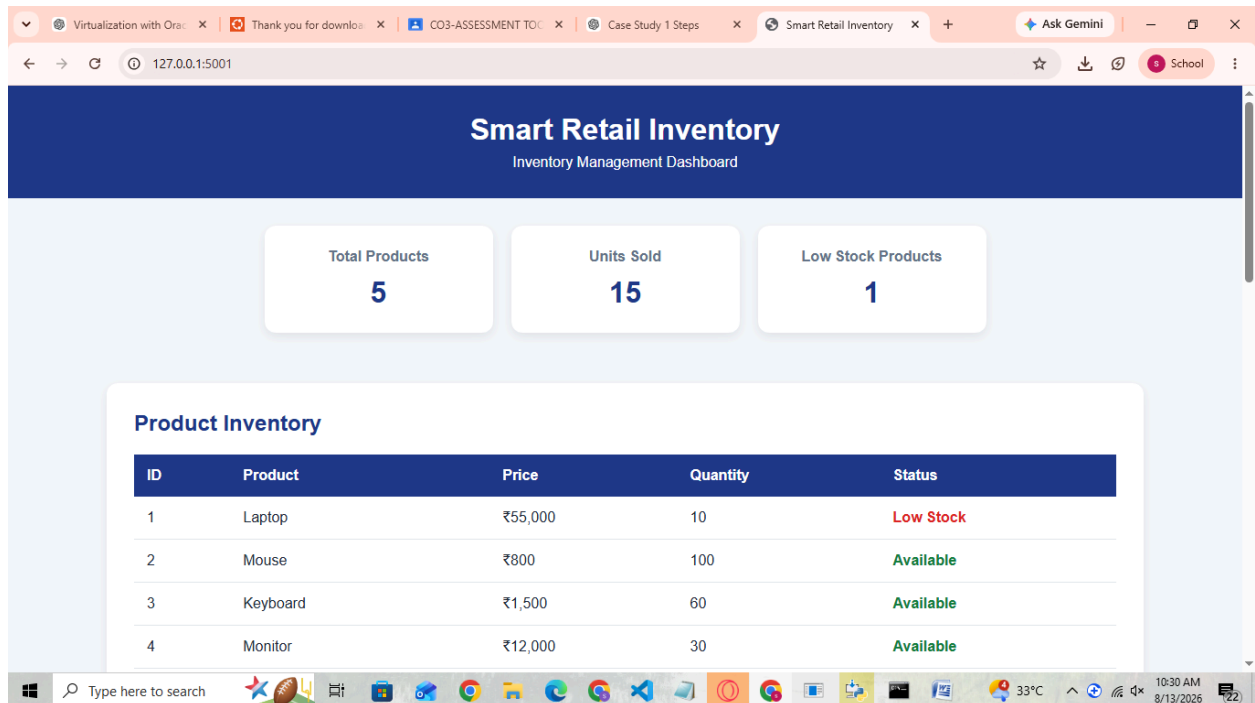
The workflow begins when a store employee or POS system sends a request to the inventory application. The request passes through the security layer consisting of VPN, firewall and API Gateway. The API Gateway forwards the request to the appropriate cloud application service. For a product or sales operation, the backend accesses the required information from cloud storage. Sales transactions are recorded as transaction logs. The stored data can then be processed by analytics and demand forecasting services. The resulting inventory information, analytics and forecasts are returned to the dashboard for users to monitor and manage stock efficiently.

Cloud Services Used

The system uses cloud-based application, storage and platform services to provide centralized inventory management. Cloud application services handle product management, sales processing, inventory operations and transaction recording. Cloud storage maintains product information, transaction records and analytics datasets. Platform services provide analytics and demand forecasting capabilities. These services allow the system to process data centrally and make inventory information available to authorized users.

Screenshot 1 — Inventory Dashboard

The Smart Retail Inventory dashboard provides a centralized interface for monitoring products and stock levels. It displays the total number of products, total units sold and the number of low-stock products. The product inventory table provides details such as product ID, product name, price, available quantity and stock status.



Screenshot 2 — Product Sale

The product sales module allows the user to sell a selected product by entering its product ID and required quantity. JavaScript sends the sale information to the Flask /sell REST API using a POST request. The backend validates the request, checks the available stock, updates the product quantity and records the transaction in the transaction file.

The screenshot displays a web application interface for a smart retail inventory system. The browser's address bar shows the URL 127.0.0.1:5001. The main content area features a table with product details and a form for selling products.

ID	Product	Price	Quantity	Status
1	Laptop	₹55,000	10	Low Stock
2	Mouse	₹800	100	Available
3	Keyboard	₹1,500	60	Available
4	Monitor	₹12,000	30	Available
5	Headphones	₹2,000	75	Available

Sell Product

Product ID Quantity

Sale completed successfully

Inventory Analytics

Screenshot 3 — Analytics

The Inventory Analytics section provides a summary of the current inventory and sales activity. The information is retrieved from the Flask /analytics API endpoint. It displays the total number of products, total units sold and products that require attention because of low stock.

The screenshot shows a web application interface with a browser window. The browser has several tabs open, including 'Smart Retail Inventory'. The address bar shows '127.0.0.1:5001'. The main content area is divided into three sections:

- Inventory Table:** A table with 5 columns: ID, Product Name, Price, Quantity, and Status. It lists three items: Keyboard (ID 3, Price ₹1,500, Quantity 60, Status Available), Monitor (ID 4, Price ₹12,000, Quantity 30, Status Available), and Headphones (ID 5, Price ₹2,000, Quantity 75, Status Available).
- Sell Product:** A form with 'Product ID' and 'Quantity' input fields, a 'Sell Product' button, and a message 'Sale completed successfully'.
- Inventory Analytics:** A summary section with three metrics: 'Total Products: 5', 'Total Units Sold: 15', and 'Low Stock: Laptop'.

The Windows taskbar at the bottom shows the time as 10:32 AM on 8/13/2026, with a temperature of 33°C.

ID	Product Name	Price	Quantity	Status
3	Keyboard	₹1,500	60	Available
4	Monitor	₹12,000	30	Available
5	Headphones	₹2,000	75	Available

Sell Product

Product ID: Enter Product ID Quantity: Enter Quantity

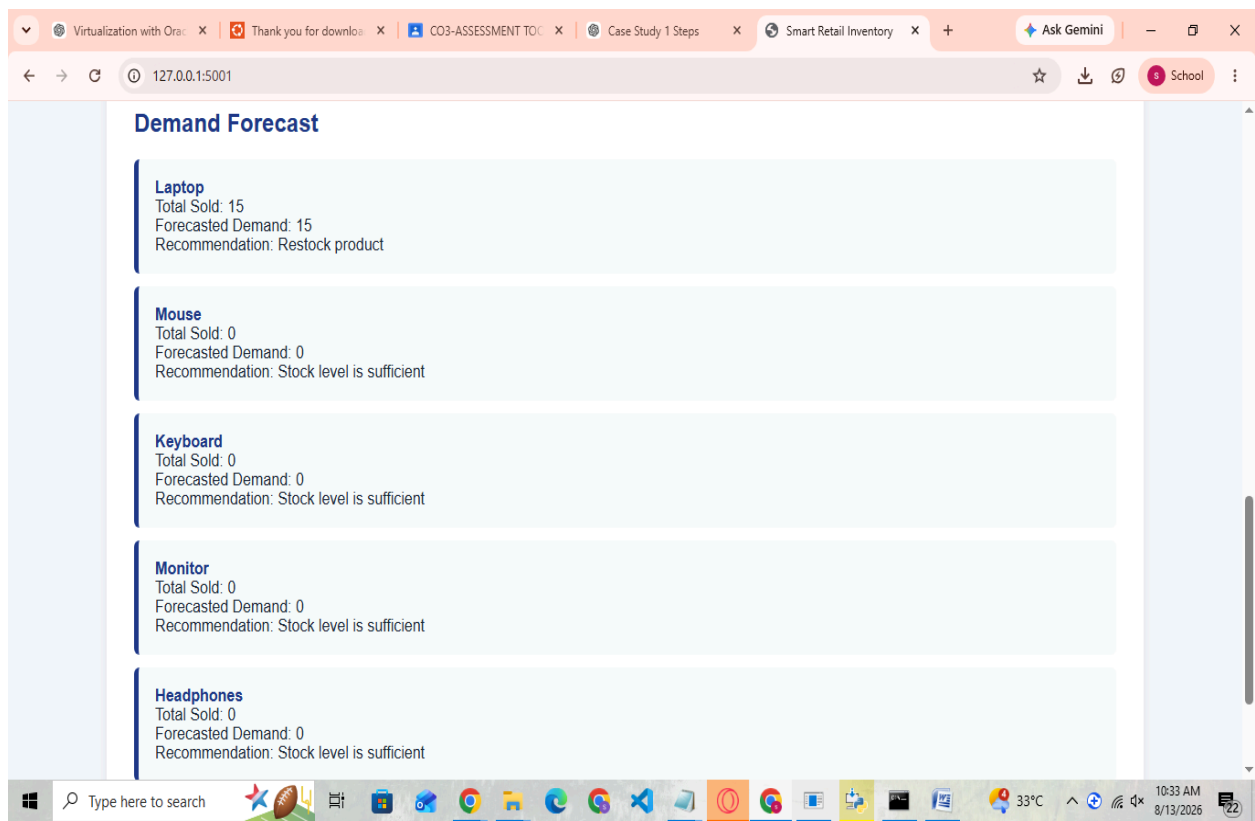
Sale completed successfully

Inventory Analytics

Total Products: 5 Total Units Sold: 15 Low Stock: Laptop

Screenshot 4 — Demand Forecast

The Demand Forecast section provides a simple prediction of product demand based on recorded sales transactions. JavaScript retrieves the forecast information from the Flask /forecast API endpoint. The webpage displays the product name, total units sold, forecasted demand and a recommendation such as monitoring or restocking the product.



PART 3 — REST API TESTING USING POSTMAN

Testing Home API

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with two items: 'GET Get data' and 'POST Post data'. The main workspace shows a GET request to 'http://127.0.0.1:5001/'. The response is a 200 OK status with a 7 ms response time and 3.06 KB of data. The response body is displayed in HTML format, showing the following code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7   <title>Smart Retail Inventory</title>
8
9   <link rel="stylesheet" href="/static/style.css">
10 </head>
11
12 <body>
```

On the right side, there is a sidebar with an AI chatbot interface. The chatbot is named 'onboard-user' and is providing assistance. The chat history shows the following messages:

- AI: data to the endpoint and test different scenarios.
- User: 3. Run the whole collection — run all requests in My Collection at once to test a full workflow.
- AI: 4. Set up a monitor — schedule your collection to run automatically and get notified of failures.
- User: What would you like to do?
- AI: What would you like to do next?
- User: Add more test scripts
- AI: Add query parameters
- User: Run the collection
- AI: Set up a monitor
- User: Describe what you need. Press @ for context, / for Skills.

The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 11:48 AM on 8/13/2026.

Testing Product API

The screenshot displays the Postman API client interface. The main workspace shows a GET request to the endpoint `http://127.0.0.1:5001/products`. The request has been successfully executed, returning a `200 OK` status with a response time of 9 ms and a body size of 587 B. The response body is displayed in JSON format, showing a list of three products:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 55000,
  "quantity": 10
},
{
  "id": 2,
  "name": "Mouse",
  "price": 800,
  "quantity": 100
},
{
  "id": 3,
  "name": "Keyboard",
  "price": 1500,
  "quantity": 60
}
```

The interface includes a sidebar on the left with a 'COLLECTIONS' panel showing 'My Collection' and 'Get data'. The bottom status bar indicates the system is connected to Git and all systems are go. The Windows taskbar at the bottom shows the time as 11:49 AM on 8/13/2025.

Testing Analytics API

GET Get data

My Collection > Get data

GET http://127.0.0.1:5001/analytics

Send

Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body > Preview Visualize

200 OK · 7 ms · 261 B · Save Response

```
{
  "low_stock_products": [
    {
      "total_products": 5,
      "total_units_sold": 15
    }
  ]
}
```

ENVIRONMENTS DOCUMENTS SPECS MOCKS FLOWS

Connect Git Terminal Console Add tests in Agent Mode

API Testing for O... /onboard-user Help me test an API I'm working with.

data to the endpoint and test different scenarios.

3. Run the whole collection — run all requests in My Collection at once to test a full workflow.

4. Set up a monitor — schedule your collection to run automatically and get notified of failures.

What would you like to do?

What would you like to do next?

Add more test scripts

Add query parameters

Run the collection

Set up a monitor

Describe what you need. Press @ for context, / for Skills.

Auto

Testing Demand Forecast API

GET Get data

My Collection > Get data

GET http://127.0.0.1:5001/forecast

Send

Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body > Preview Visualize

200 OK · 9 ms · 937 B · Save Response

```
{
  "forecast": [
    {
      "forecasted_demand": 15,
      "product_name": "Laptop",
      "recommendation": "Restock product",
      "total_sold": 15
    },
    {
      "forecasted_demand": 0,
      "product_name": "Mouse",
      "recommendation": "Stock level is sufficient",
      "total_sold": 0
    },
    {
      "forecasted_demand": 0,
      "product_name": "Keyboard",
      "recommendation": "Stock level is sufficient",
      "total_sold": 0
    },
    {
      "forecasted_demand": 0,
      "product_name": "Monitor",
      "recommendation": "Stock level is sufficient",
      "total_sold": 0
    },
    {
      "forecasted_demand": 0,
      "product_name": "Headphones",
      "recommendation": "Stock level is sufficient",
      "total_sold": 0
    }
  ]
}
```

ENVIRONMENTS DOCUMENTS SPECS MOCKS FLOWS

Connect Git Terminal Console Add tests in Agent Mode

API Testing for O... /onboard-user Help me test an API I'm working with.

data to the endpoint and test different scenarios.

3. Run the whole collection — run all requests in My Collection at once to test a full workflow.

4. Set up a monitor — schedule your collection to run automatically and get notified of failures.

What would you like to do?

What would you like to do next?

Add more test scripts

Add query parameters

Run the collection

Set up a monitor

Describe what you need. Press @ for context, / for Skills.

Auto

REPORT WITH SCREENSHOTS AND EXPLANATION

Introduction

The Smart Retail Inventory Management System is a web-based application developed to manage retail product inventory efficiently. The system allows users to view available products, record product sales, monitor inventory levels, analyze sales information, and generate demand forecasts. The application uses HTML, CSS and JavaScript for the frontend, Flask/Python for the backend REST API, and a structured inventory data layer for storing and processing product information.

Objectives

The main objectives of the Smart Retail Inventory Management System are:

1. To provide a simple interface for managing retail inventory.
2. To display available products and their stock information.
3. To provide REST APIs for inventory operations.
4. To record product sales and update inventory information.
5. To identify low-stock products using analytics.
6. To generate demand forecasts based on sales information.
7. To demonstrate API functionality using Postman.

Conclusion

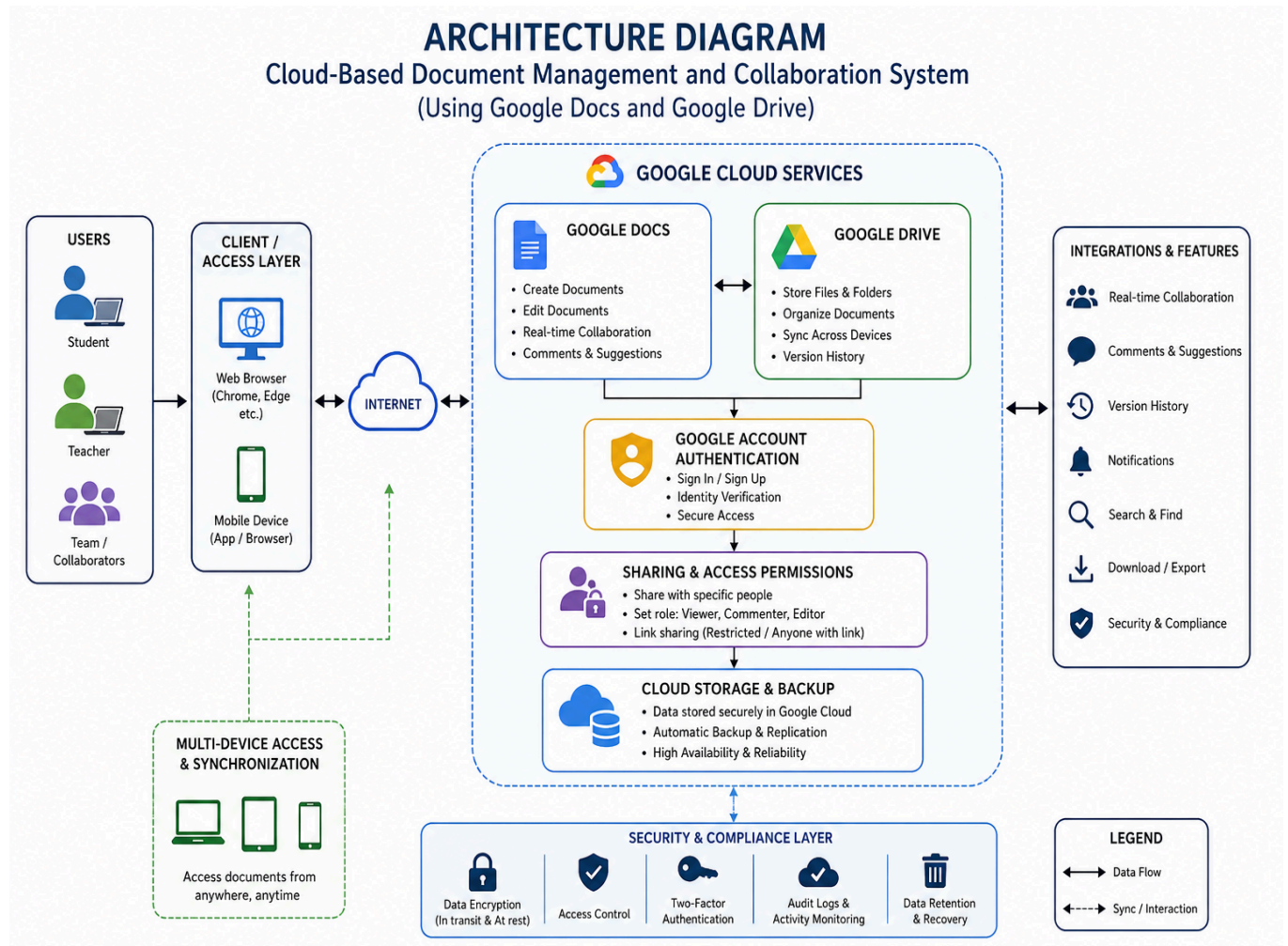
The Smart Retail Inventory Management System was successfully developed and tested. The system provides a web-based interface for inventory management and a Flask-based REST API for handling inventory operations. The APIs were successfully tested using Postman,

including product retrieval, sales processing, analytics, and demand forecasting. The project demonstrates how frontend technologies, backend REST APIs, data processing, and API testing can be integrated to create a basic smart retail inventory management solution.

CASE STUDY-2

1. ARCHITECTURE DIAGRAM

Cloud-Based Document Management and Collaboration System



The proposed system is a cloud-based document management and collaboration system using Google Docs and Google Drive. Users access the system through a web browser or mobile application over the Internet. Google Account authentication is used to securely identify users. Google Docs provides facilities for creating, editing and collaborating on documents, while Google Drive provides cloud-based storage for documents and files. Sharing and access permissions control which users can view, comment on or edit the documents. The documents are

stored in cloud storage, providing convenient access, synchronization and backup from different devices.

2. COMPONENT IDENTIFICATION

S.No.	Component	Purpose / Function
1	Users	Students, teachers, and team members who create, access, edit, and share documents.
2	Client / Access Layer	Provides access to the cloud system through a web browser or mobile application.
3	Internet	Provides communication between the user's device and cloud services.
4	Google Docs	Used to create, edit, format, comment on, and collaborate on documents in real time.
5	Google Drive	Provides cloud storage for documents, files, and folders and allows them to be organized and accessed from different devices.
6	Google Account Authentication	Verifies the identity of users and provides secure access to Google services.
7	Sharing & Access Permissions	Controls who can view, comment on, or edit a document.
8	Cloud Storage & Backup	Stores documents in the cloud and supports availability, synchronization, and recovery

Component Description

The system consists of multiple components that work together to provide cloud-based document management and collaboration. Users access Google services through web browsers or mobile applications over the Internet. Google Docs is responsible for document creation, editing, and real-time collaboration, while Google Drive provides cloud-based file storage and organization. Google Account authentication ensures that only authenticated users can access the system. Sharing and access permissions control the operations that users can perform on documents. Cloud storage provides availability and synchronization across devices, while the security layer protects documents and user accounts through authentication, access control, encryption, and activity monitoring.

3. WORKFLOW EXPLANATION

User



Open Web Browser / Mobile App



Connect to Internet



Step 1: User Access

The user accesses the cloud-based document management system using a computer, laptop, tablet, or smartphone.

Step 2: Internet Connection

The user's device connects to Google cloud services through the Internet.

Step 3: Authentication

The user signs in using a Google Account. Authentication verifies the user's identity before providing access to Google Docs and Google Drive.

Step 4: Open Google Docs

After authentication, the user opens Google Docs and creates a new document or opens an existing document.

Step 5: Create and Edit Document

The user can enter text, format the document, add information, and make changes. Multiple authorized users can collaborate on the same document.

Step 6: Store the Document

The document is stored in Google Drive, which provides cloud-based storage and organization using files and folders.

Step 7: Configure Permissions

The document owner can specify access permissions for other users. Depending on the permission granted, users can be allowed to **view, comment, or edit** the document.

Step 8: Collaboration

The document can be shared with teachers, students, or team members. Authorized collaborators can access the document and work on it according to their assigned permissions.

Step 9: Synchronization

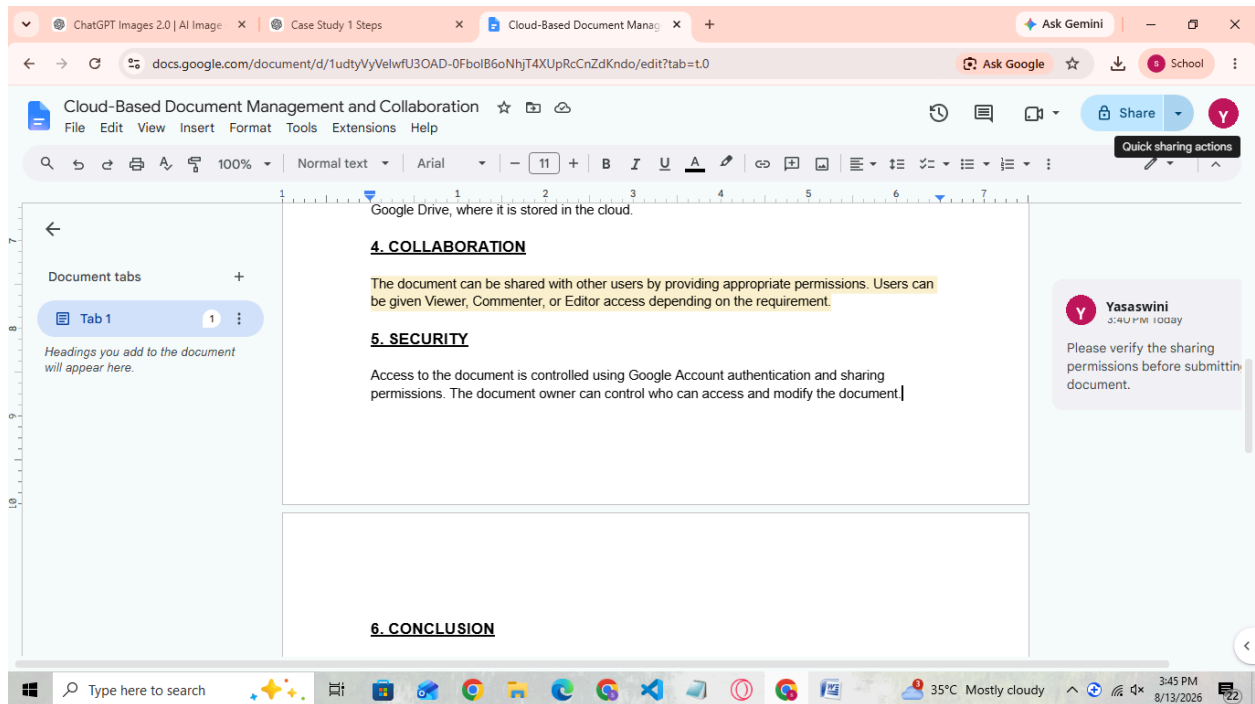
Changes made to the document are synchronized through the cloud. Authorized users can see the latest version when accessing the document from supported devices.

Step 10: Cloud Storage and Recovery

The document remains stored in the cloud, allowing authorized users to access it when required. Features such as version history can also help track previous changes.

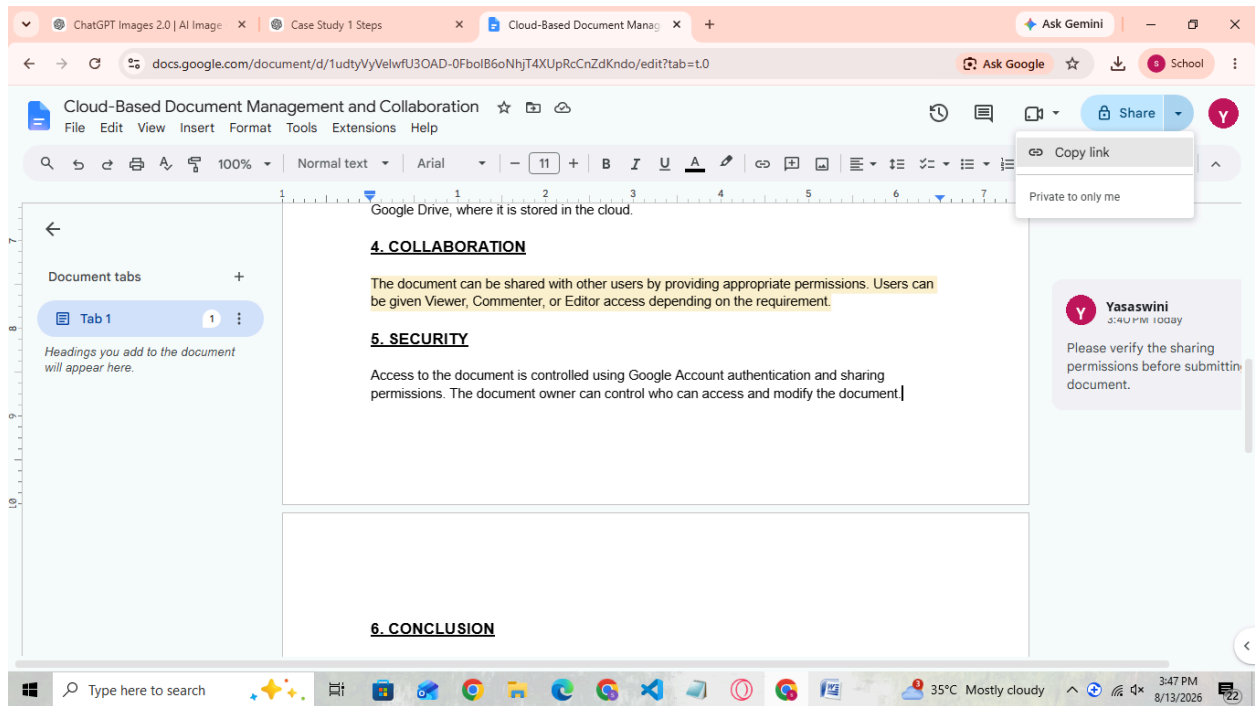
GOOGLE DOCS PRACTICAL

Adding a Comment

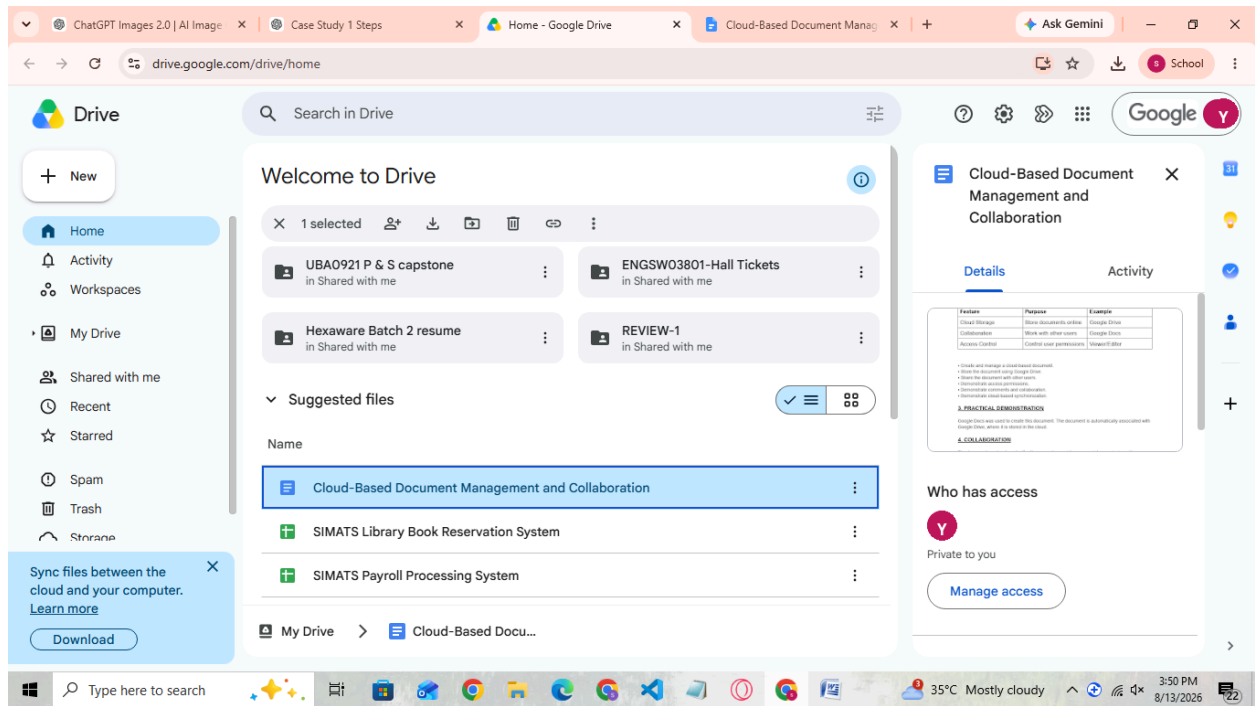


sharing and Access Permissions

<https://docs.google.com/document/d/1udtyVyVelwfU3OAD-0FbolB6oN hjT4XUpRcCnZdKndo/edit?usp=sharing>

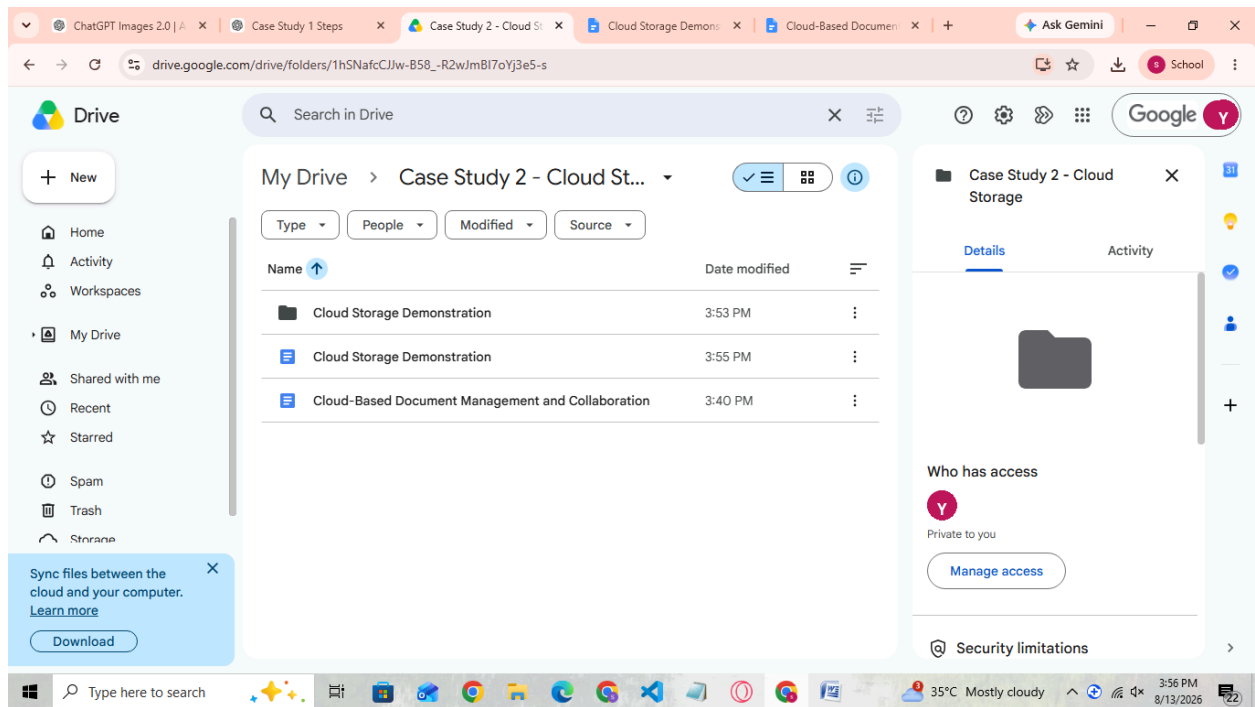


Document Stored in Google Drive

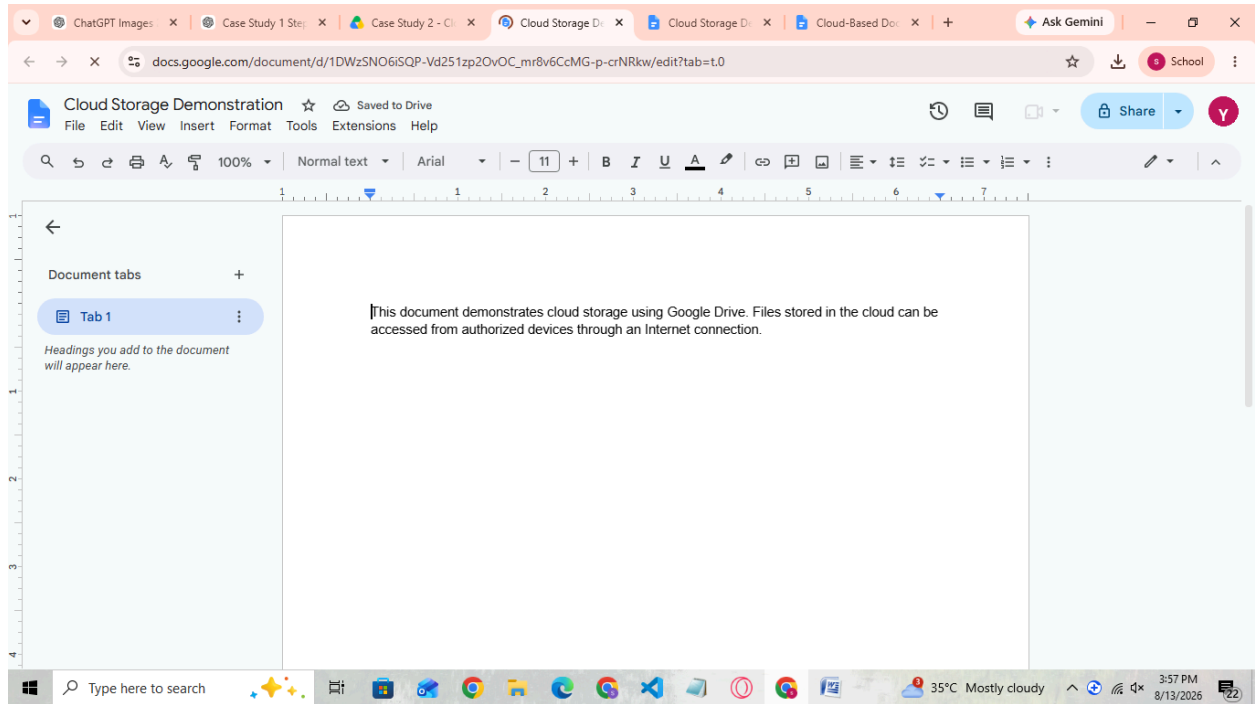


CLOUD STORAGE DEMONSTRATION

Cloud Storage Folder Creation



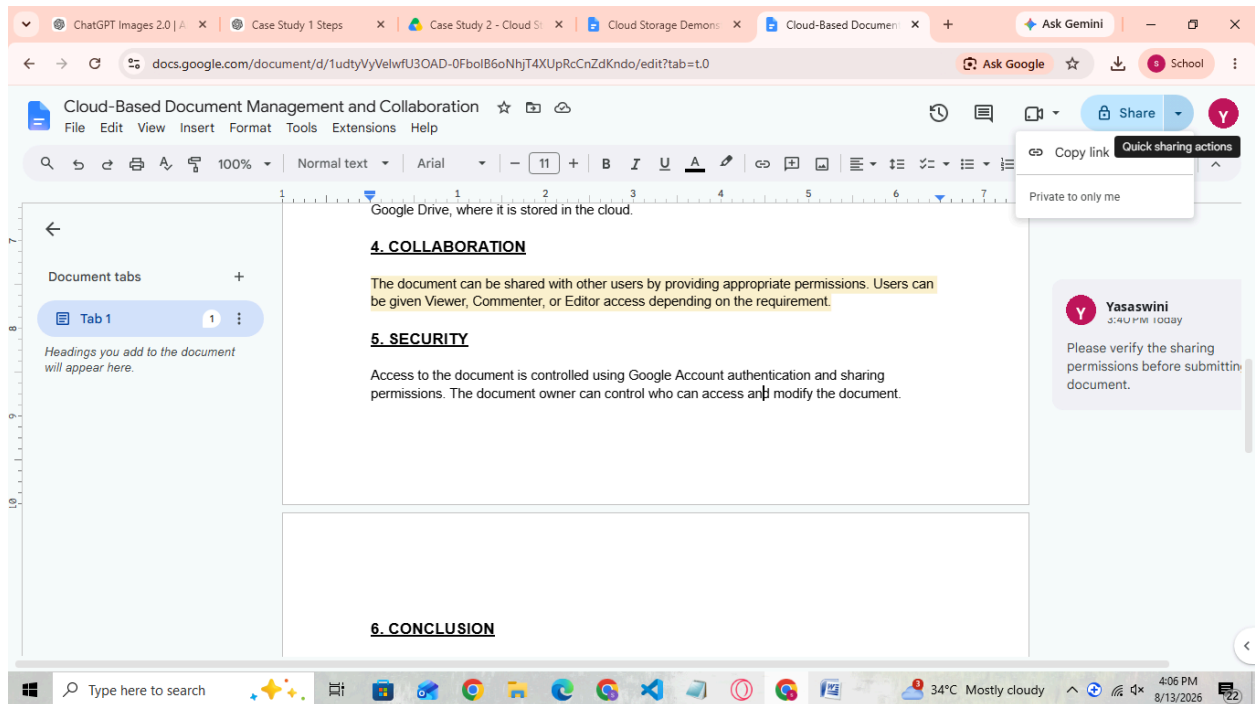
Demonstrate file access



SECURITY ANALYSIS

Security Analysis Explanation

Security is an important part of cloud-based document management. Google Account authentication helps verify the identity of users. Access permissions allow document owners to control who can view, comment on, or edit documents. Restricted sharing should be preferred when documents contain private or academic information. Encryption helps protect information while it is transmitted and stored. Version history can help identify previous changes and restore earlier versions when required. Users should also follow good security practices such as using strong passwords, enabling additional account protection where available, and avoiding unnecessary public sharing.

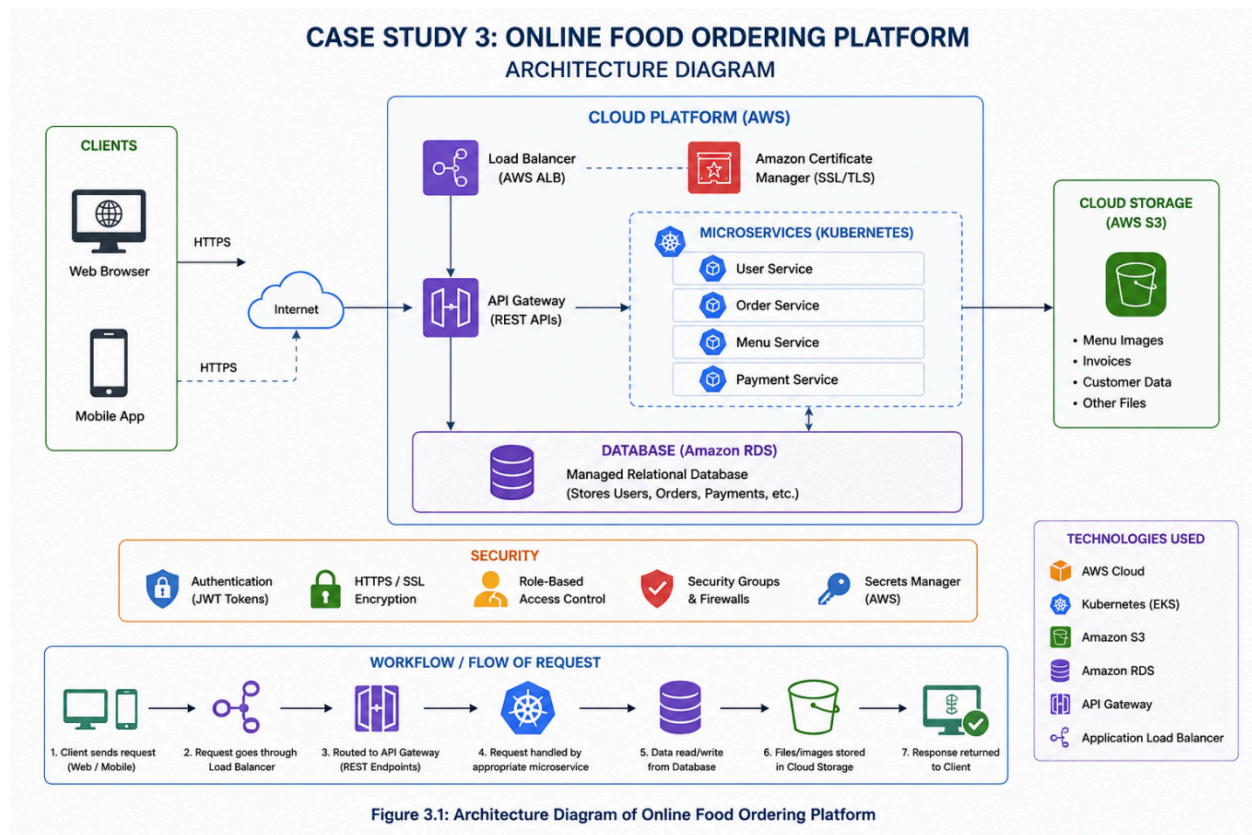


CONCLUSION

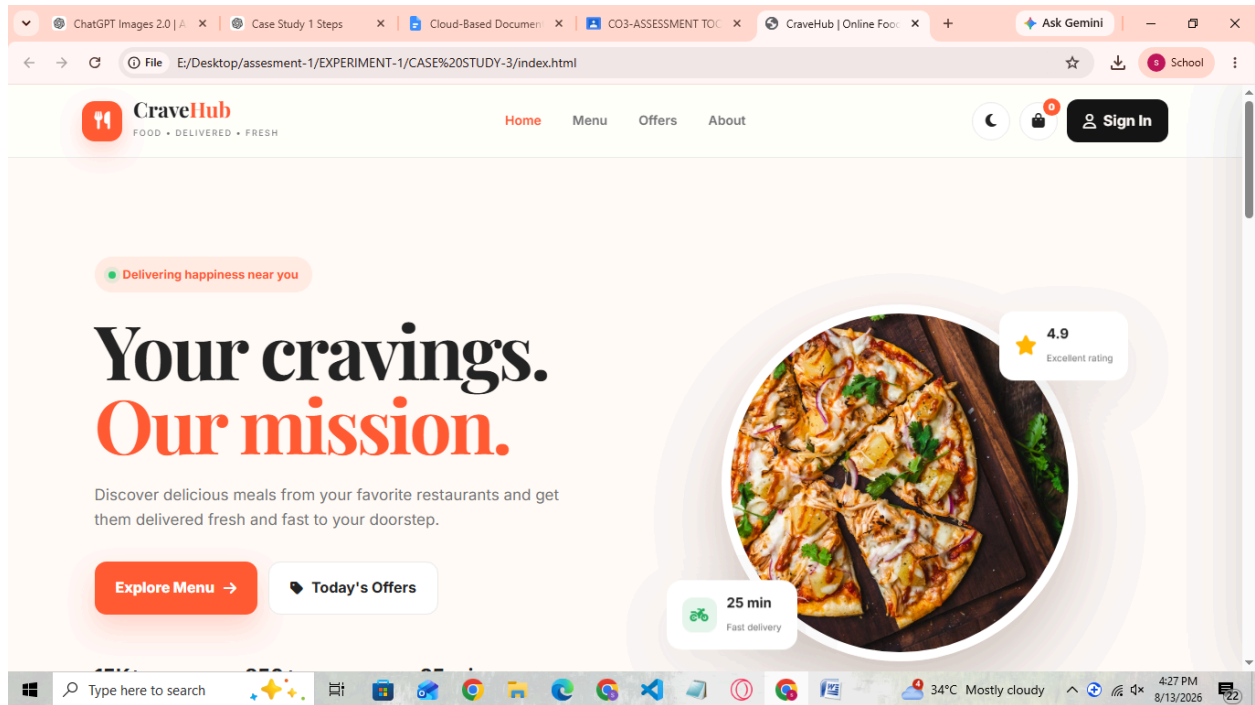
The Cloud-Based Document Management and Collaboration System was successfully demonstrated using Google Docs and Google Drive. The system provides cloud-based document creation, storage, sharing, and collaboration. Users can work on documents and communicate through comments while access permissions control who can view, comment, or edit the documents. Google Drive provides centralized cloud storage and organization of project files. Security mechanisms such as account authentication, restricted sharing, access permissions, encryption, and version history help protect documents and maintain controlled access. Overall, the case study demonstrates how cloud computing can improve document management, accessibility, collaboration, and secure file storage.

CASE STUDY 3

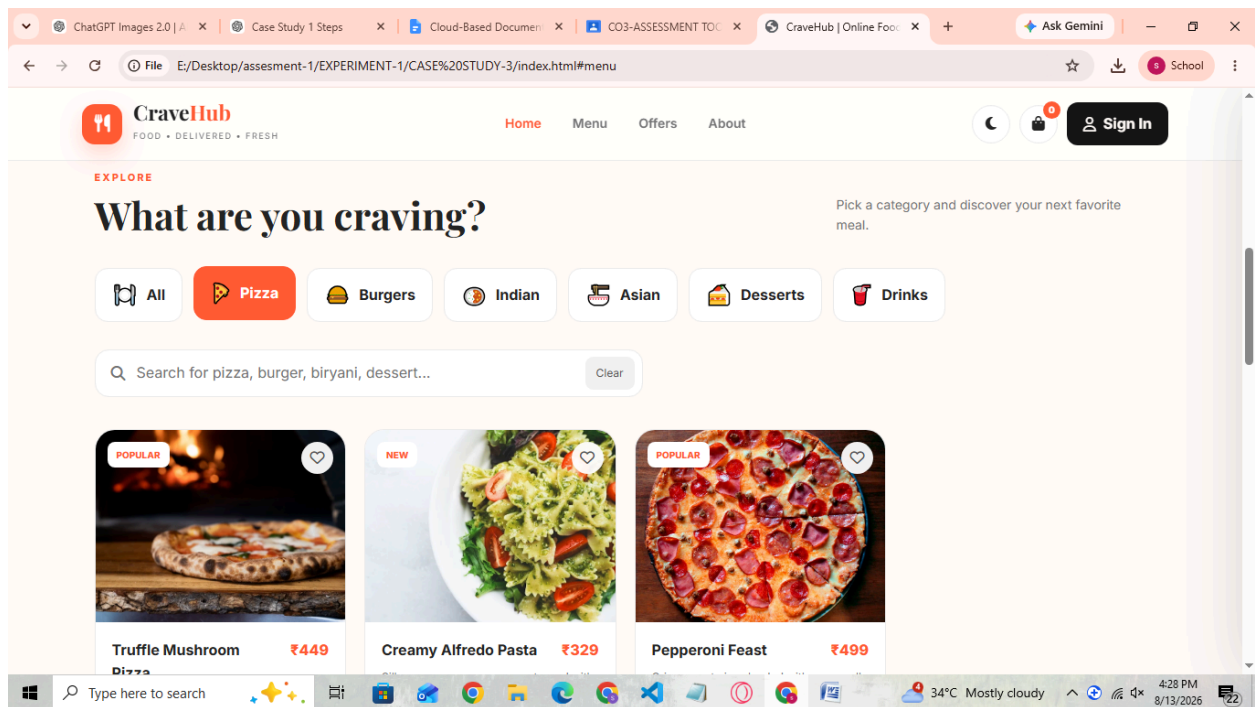
ARCHITECTURE DIAGRAM



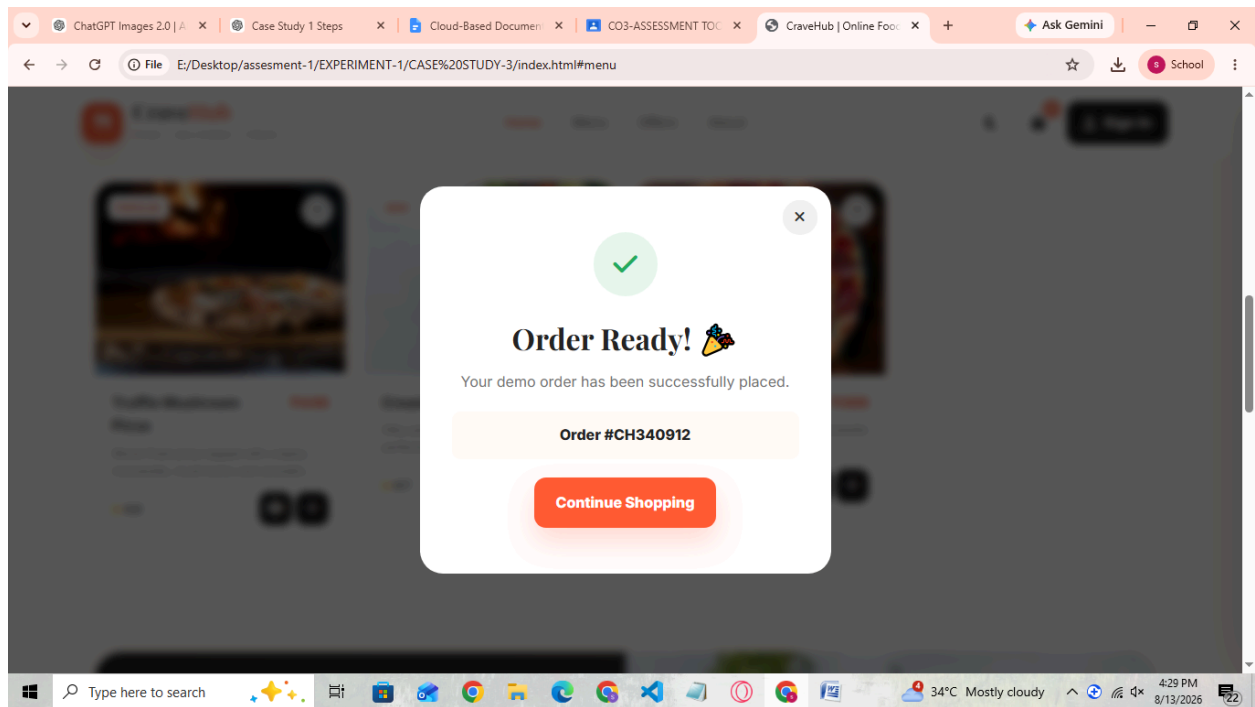
Home Page

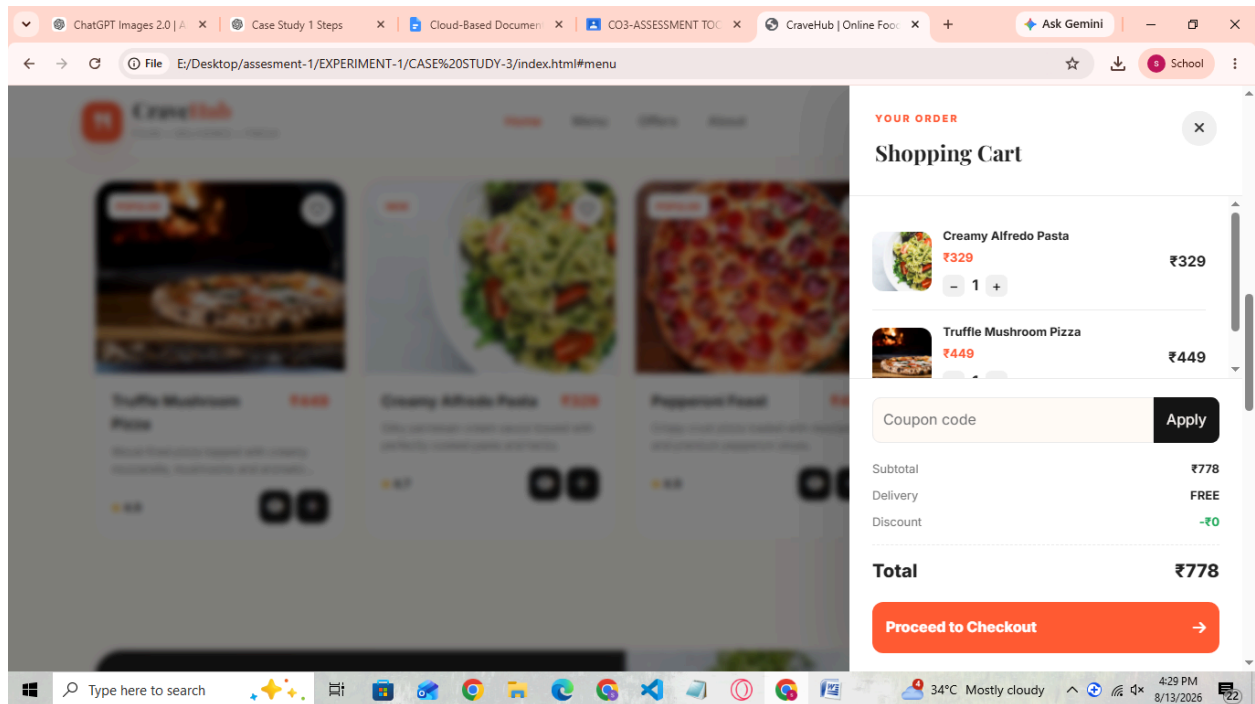


Menu



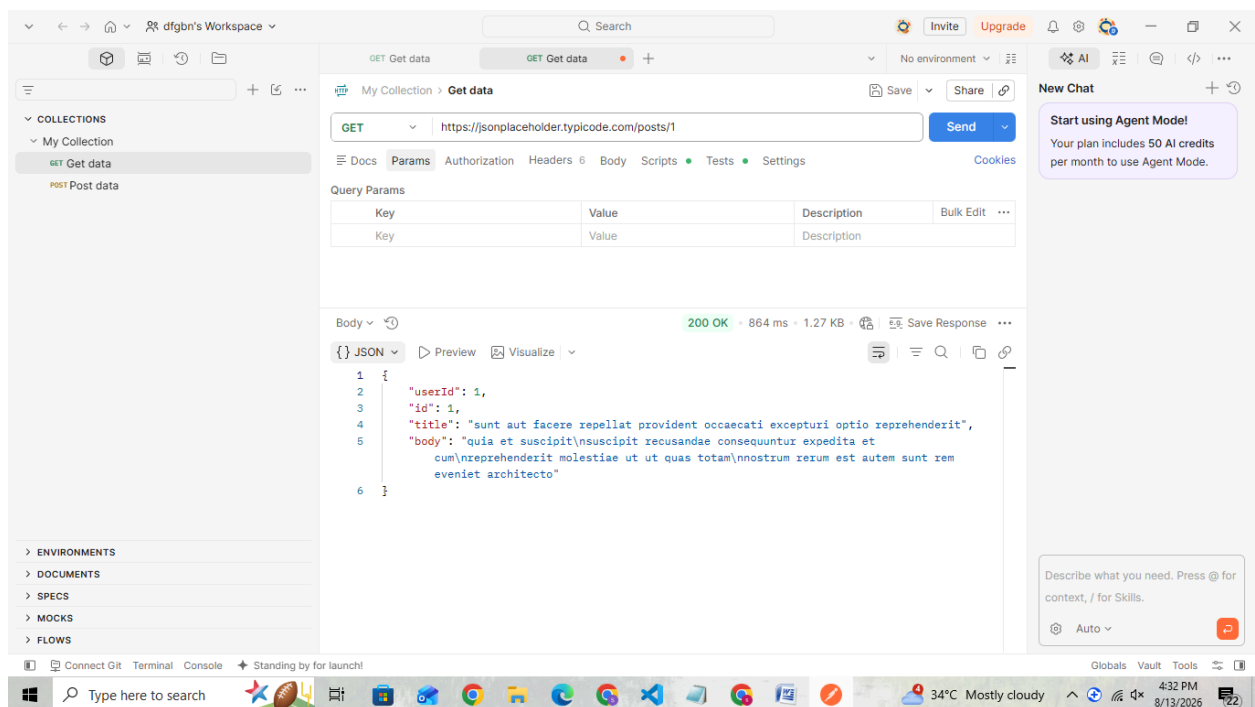
Cart





REST API TESTING WITH POSTMAN

Create a GET request



Test POST

The screenshot shows the Postman interface with a POST request to `https://jsonplaceholder.typicode.com/posts`. The response is `201 Created` with a status of `569 ms` and a body size of `1.2 KB`. The response body is a JSON object: `{ "id": 101 }`. The left sidebar shows the 'My Collection' with 'Get data' and 'Post data' items. The right sidebar shows a 'New Chat' section with a message about AI credits and a 'Fix failing status code test' button.

Key	Value	Description	Bulk Edit
Key	Value	Description	

```
{ "id": 101 }
```

201 Created - 569 ms - 1.2 KB

Test script

```
1 - pm.test("Stat  
code is 200",  
function () {  
2 -  
3 - pm.response.t  
1 + pm.test("Stat  
code is 201",  
function () {  
2 +
```

Apply

Complete the POST payload

Describe what you need. Press @ for context, / for Skills.

Auto

Test PUT

The screenshot shows the Postman interface with a PUT request to `https://jsonplaceholder.typicode.com/posts/1`. The response is `200 OK` with a status of `3.83 s` and a body size of `1.08 KB`. The response body is a JSON object: `{ "id": 1 }`. The left sidebar shows the 'My Collection' with 'Get data' and 'Post data' items. The right sidebar shows a 'New Chat' section with a message about AI credits and a 'Fix failing status code test' button.

Key	Value	Description	Bulk Edit
Key	Value	Description	

```
{ "id": 1 }
```

200 OK - 3.83 s - 1.08 KB

Test script

```
1 - pm.test("Stat  
code is 200",  
function () {  
2 -  
3 - pm.response.t  
1 + pm.test("Stat  
code is 201",  
function () {  
2 +
```

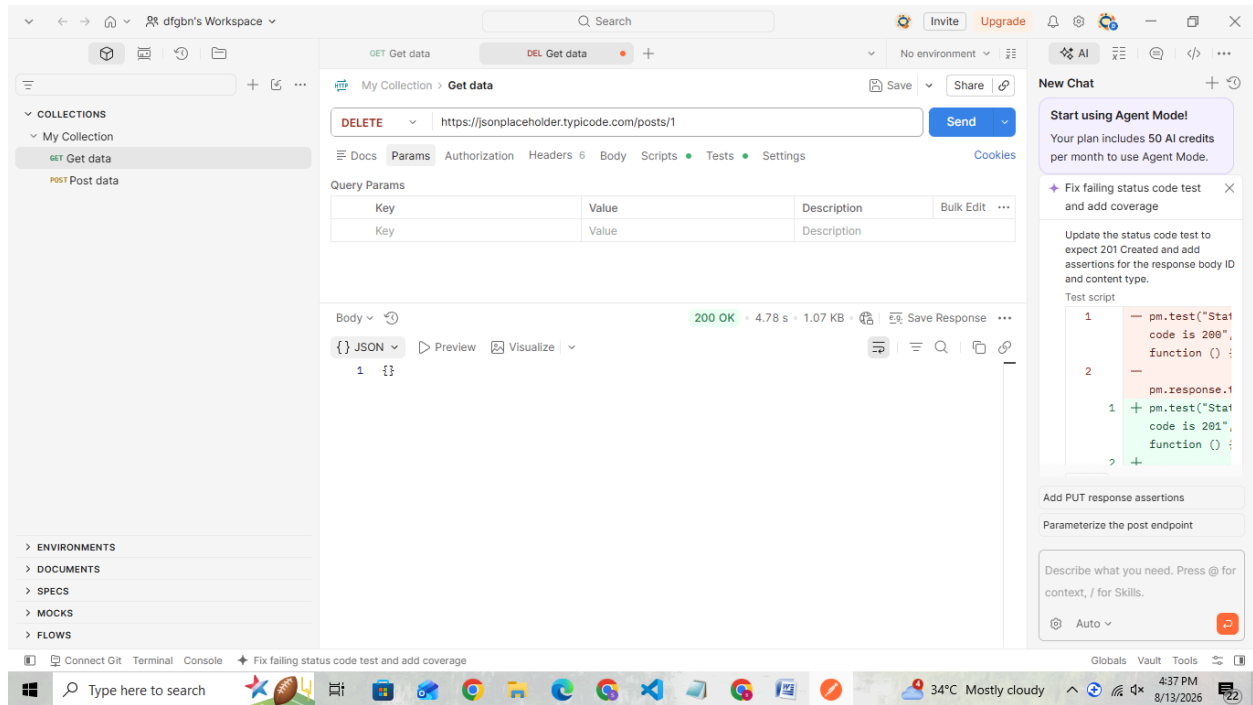
Apply

Complete the POST payload

Describe what you need. Press @ for context, / for Skills.

Auto

Test DELETE



STEP 4 — SECURITY OF THE ONLINE FOOD ORDERING PLATFORM

4.1 Authentication Using Tokens

The food ordering platform uses token-based authentication to verify users before allowing them to access protected services. After successful login, the server generates an authentication token. The client sends this token with subsequent API requests. The backend verifies the token before processing the request. This prevents unauthorized users from accessing protected resources.

4.2 HTTPS Security

HTTPS is used to protect communication between the web browser, mobile clients, and backend REST APIs. HTTPS encrypts data

transmitted over the network and helps prevent attackers from reading or modifying sensitive information such as login credentials, customer information, and order details.

4.3 Role-Based Access Control

Role-Based Access Control restricts access to system resources according to the user's assigned role. Different users are given different permissions based on their responsibilities.

4.4 Secure Cloud Storage

Cloud storage is used to store menu images, invoices, and customer-related data. Access to stored resources is controlled using authentication and authorization policies. Sensitive information should not be publicly exposed. Encryption and access-control mechanisms help protect stored data from unauthorized access.

Results and Observations

The online food ordering platform successfully demonstrates the integration of a web-based client with RESTful backend services. The webpage provides an interactive food ordering interface, while the REST APIs support retrieving, creating, updating, and deleting order information. The architecture demonstrates the use of cloud infrastructure, storage, virtualization, and Kubernetes for scalability and reliability. Security mechanisms such as HTTPS, authentication tokens, and role-based access control provide protection for users and resources.

Test	Method	Result
Retrieve data	GET	Successful
Create order	POST	Successful
Update order	PUT	Successful
Delete order	DELETE	Successful
Webpage	HTML/CSS/JS	Successfully demonstrated

Test	Method	Result
Security	HTTPS/Token/RBAC	Designed

Conclusion

The Online Food Ordering Platform demonstrates how modern cloud technologies can be integrated to build a scalable, reliable, and secure application. The system combines a responsive web interface, RESTful APIs, cloud storage, virtual machines, and managed Kubernetes.

Postman testing demonstrates the major REST operations, including GET, POST, PUT, and DELETE. Security mechanisms such as HTTPS, token-based authentication, and role-based access control help protect the platform and its resources.